05_iceberg_catalog

August 18, 2026

1 NB5 — Apache Iceberg & the Catalog as Control Plane

Stack: `pyiceberg` + a local SQLite catalog. No JVM, no server, no cloud. Maps to slide §4 (Apache Iceberg) + §12 (Catalog = Control Plane) + deliverable bullet 5.

NB1–NB4 used Delta. Here you meet the *other* open table format — and more importantly, the thing both formats now agree is the centre of the architecture: **the catalog**.

> **The 2026 shift.** Iceberg 1.11 moved scan planning *server-side*; Delta 4.1 shipped *catalog-managed tables*. Two rival camps, one conclusion: the catalog stopped being a name→path lookup and became the **query planner and security boundary**. That is the single biggest architectural change of the year, and it is what this notebook makes concrete.
>
> Production equivalent: `SqlCatalog(sqlite)` → `load_catalog(type="rest")` against Polaris / Unity / Lakekeeper / Glue. **Same API, same on-disk metadata** — you are not learning a toy dialect.

[1]:
```
import _setup  # noqa: F401  -- adds scripts/ to sys.path (file-relative)

import datetime as dtm

import pyarrow as pa

from lakehouse import catalog, count_files, du, human, namespace, reset_catalog

CAT = "nb5"           # own catalog dir: NB6/NB8/`make smoke` cannot disturb it
reset_catalog(CAT)    # idempotent rerun
cat = catalog(CAT)
ns = namespace(cat, "lake")
print(f"Catalog: {type(cat).__name__}   namespaces: {cat.list_namespaces()}")
```

```
Catalog: SqlCatalog   namespaces: [('lake',)]
```

1.1 1. Create a table *through the catalog*

Note what you do **not** do: you never pick a path. The catalog owns the layout. That indirection is exactly what lets the catalog later vend credentials, enforce row filters, and plan scans on your behalf.

`nullable=False` is not cosmetic — Iceberg tracks *required* vs *optional* per field, and a required field can never be silently dropped.

[2]:
```
SCHEMA = pa.schema([
    pa.field("event_id",   pa.int64(),           nullable=False),
    pa.field("ts",         pa.timestamp("us"),   nullable=False),
    pa.field("model",      pa.string()),
    pa.field("latency_ms", pa.int64()),
    pa.field("cost_usd",   pa.float64()),
])

tbl = cat.create_table(f"{ns}.llm_events", schema=SCHEMA)
print(f"Created {tbl.name()}")
print(f"  location:  {tbl.location()}")
print(f"  metadata:  {tbl.metadata_location.rsplit('/', 1)[-1]}")
print(f"  format-v{tbl.format_version}")
```

```
Created ('lake', 'llm_events')
  location:  file://D:\AI20K\Track2\Day18-Track2-Lakehouse-
Lab\_lakehouse\iceberg\nb5\warehouse/lake/llm_events
  metadata:  00000-4b7becf9-66df-4ae9-a2ee-cbda62d05852.metadata.json
  format-v2
```

1.2 2. Hidden partitioning — the feature that killed Hive

In Hive you partitioned by a **derived column** (`dt=2026-08-05`) that the user had to know about and filter on by hand. Forget it, and you full-scan the table. Every data team has that outage story.

Iceberg stores the *transform* (`day(ts)`) in metadata instead. You filter on `ts` — the real column — and the engine derives the partition itself.

[3]:
```
from pyiceberg.transforms import DayTransform  # noqa: E402

with tbl.update_spec() as spec:
    spec.add_field("ts", DayTransform(), "ts_day")

tbl = cat.load_table(f"{ns}.llm_events")  # refresh after a metadata change
print(f"Partition spec: {tbl.spec()}")
print("\nNote: 'ts_day' is NOT a column you insert. It is derived from ts.")
```

```
Partition spec: [
  1000: ts_day: day(2)
]

Note: 'ts_day' is NOT a column you insert. It is derived from ts.
```

1.3 3. Append 10 daily batches

Each `append()` is one atomic commit → one snapshot. This is the same ACID guarantee Delta gives you, expressed through a different metadata tree.

[4]:
```
MODELS = ["claude-haiku-4-5", "claude-sonnet-4-6", "claude-opus-4-7"]
COST_PER_CALL = {"claude-haiku-4-5": 0.0004, "claude-sonnet-4-6": 0.003,␣
 ↪"claude-opus-4-7": 0.015}
ROWS_PER_DAY = 500
N_DAYS = 10


def day_batch(day: int) -> pa.Table:
    """One day of synthetic inference traffic."""
    models = [MODELS[i % 3] for i in range(ROWS_PER_DAY)]
    return pa.table({
        "event_id":   [day * ROWS_PER_DAY + i for i in range(ROWS_PER_DAY)],
        "ts":         [dtm.datetime(2026, 8, day, 3, i % 60) for i in␣
 ↪range(ROWS_PER_DAY)],
        "model":      models,
        "latency_ms": [200 + (i * 7) % 3000 for i in range(ROWS_PER_DAY)],
        "cost_usd":   [COST_PER_CALL[m] for m in models],
    }, schema=SCHEMA)


for day in range(1, N_DAYS + 1):
    tbl.append(day_batch(day))

tbl = cat.load_table(f"{ns}.llm_events")
print(f"Rows:      {tbl.scan().to_arrow().num_rows:,}")
print(f"Snapshots: {len(tbl.snapshots())}   (one per commit)")
print(f"Data files: {tbl.inspect.files().num_rows}")
```

```
Rows:      5,000
Snapshots: 10   (one per commit)
Data files: 10
```

1.4 4. Scan planning — the number the catalog computes for you

`plan_files()` is the planning step itself: given a filter, which data files must actually be read? In Iceberg 1.11+ this can run **inside the catalog server**, so a laptop client never downloads the manifest tree at all.

Watch: we filter on **`ts`**, never on `ts_day`.

[5]:
```
scan_all = tbl.scan()
scan_one_day = tbl.scan(row_filter="ts >= '2026-08-05T00:00:00' and ts <␣
 ↪'2026-08-06T00:00:00'")
```

```
files_all = len(list(scan_all.plan_files()))
files_one = len(list(scan_one_day.plan_files()))

print(f"Files to read, no filter:    {files_all}")
print(f"Files to read, one-day filter: {files_one}")
print(f"→ Pruning ratio: {files_all / max(files_one, 1):.0f}×   (target   5×)")
print(f"  rows returned: {scan_one_day.to_arrow().num_rows:,}")

PRUNE_RATIO = files_all / max(files_one, 1)
assert PRUNE_RATIO >= 5, f"expected  5x pruning, got {PRUNE_RATIO:.1f}x"
```

```
Files to read, no filter:    10
Files to read, one-day filter: 1
→ Pruning ratio: 10×   (target   5×)
  rows returned: 500
```

1.4.1 The Hive comparison, made concrete

A Hive-style user who forgets the partition predicate reads **all** files. The Iceberg user cannot make that mistake — there is no partition column to forget. Same query intent, an order of magnitude difference in bytes scanned.

[6]:
```
# Scale the toy numbers to a realistic file size and a metered engine
# (Athena/BigQuery bill per TB scanned; $5/TB is the long-standing list price).
FILE_MB, PRICE_PER_TB, QUERIES_PER_DAY = 512, 5.0, 10_000
waste_tb = (files_all - files_one) * FILE_MB / 1_048_576

print(f"Hive user who forgets `WHERE dt=...`:  reads {files_all} files")
print(f"Iceberg user filtering on `ts`:        reads {files_one} files")
print(f"\nAt {FILE_MB} MB/file and ${PRICE_PER_TB:.0f}/TB scanned:")
print(f"  wasted per query: {waste_tb * 1024:.1f} GB  =  ${waste_tb *␣
 ↪PRICE_PER_TB:.3f}")
print(f"  × {QUERIES_PER_DAY:,} queries/day  =  ${waste_tb * PRICE_PER_TB *␣
 ↪QUERIES_PER_DAY:,.0f}/day")
print("\nThat is the bill for one forgotten predicate. Hidden partitioning␣
 ↪removes")
print("the opportunity to forget.")
```

```
Hive user who forgets `WHERE dt=…`:  reads 10 files
Iceberg user filtering on `ts`:        reads 1 files

At 512 MB/file and $5/TB scanned:
  wasted per query: 4.5 GB  =  $0.022
  × 10,000 queries/day  =  $220/day

That is the bill for one forgotten predicate. Hidden partitioning removes
the opportunity to forget.
```

1.5 5. The three-tier metadata tree

This is the structure the deck draws as a chain. Now walk it for real:

```
catalog  →  metadata.json  →  manifest list  →  manifest files  →  data files
            (schema, specs,   (one per         (file stats:
             snapshot ptr)     snapshot)        min/max, counts)
```

Every tier exists to let the tier above **skip** work below it.

[7]:
```
snaps = tbl.inspect.snapshots()
mans = tbl.inspect.manifests()
files = tbl.inspect.files()

print(f"Tier 1  metadata.json     : {tbl.metadata_location.rsplit('/', 1)[-1]}")
print(f"Tier 2  manifest lists    : {snaps.num_rows} (one per snapshot)")
print(f"Tier 3  manifest files    : {mans.num_rows}")
print(f"        data files        : {files.num_rows}")
print(f"\nMetadata log entries: {tbl.inspect.metadata_log_entries().num_rows}")
print(f"Partitions tracked  : {tbl.inspect.partitions().num_rows}")
```

```
Tier 1  metadata.json     :
00011-fc4f9c65-0554-414b-bfa1-6c4276bec498.metadata.json
Tier 2  manifest lists    : 10 (one per snapshot)
Tier 3  manifest files    : 10
        data files        : 10

Metadata log entries: 12
Partitions tracked  : 10
```

1.5.1 The cost of planning

Metadata is not free. Count what a planner would have to open, and compare metadata bytes to data bytes — this ratio is *why* Iceberg v4 is redesigning the metadata tree, and why server-side planning matters at scale.

[8]:
```
loc = tbl.location().replace("file://", "")
meta_bytes = du(f"{loc}/metadata")
data_bytes = du(f"{loc}/data")
print(f"data/     {human(data_bytes):>10}   ({count_files(f'{loc}/data')}␣
 ↪parquet files)")
print(f"metadata/ {human(meta_bytes):>10}   ({count_files(f'{loc}/metadata', '.
 ↪avro')} avro + "
      f"{count_files(f'{loc}/metadata', '.json')} json)")
print(f"→ metadata is {meta_bytes / max(data_bytes, 1) * 100:.1f}% of table␣
 ↪size")
print("\nAt 10 rows/file this looks absurd. At 512 MB/file it is ~0.1%.")
print("Small files punish you TWICE: more data files AND more metadata to plan␣
 ↪over.")
```

```
data/        47.3 KB   (10 parquet files)
metadata/   131.2 KB   (20 avro + 12 json)
→ metadata is 277.4% of table size

At 10 rows/file this looks absurd. At 512 MB/file it is ~0.1%.
Small files punish you TWICE: more data files AND more metadata to plan over.
```

1.6 6. Schema evolution by field-ID (not by name, not by position)

Parquet columns are positional. Hive matched by name. Both break under rename/reorder. Iceberg assigns every field a **permanent integer ID**; names are just labels on top of it.

So a rename is a *metadata-only* operation — zero files rewritten.

[9]:
```
from pyiceberg.types import StringType  # noqa: E402

print("Field IDs before:", [(f.field_id, f.name) for f in tbl.schema().fields])

with tbl.update_schema() as upd:
    upd.add_column("tier", StringType(), doc="customer tier, added after 5000␣
 ↪rows existed")
tbl = cat.load_table(f"{ns}.llm_events")

with tbl.update_schema() as upd:
    upd.rename_column("latency_ms", "latency_millis")
tbl = cat.load_table(f"{ns}.llm_events")

print("Field IDs after :", [(f.field_id, f.name) for f in tbl.schema().fields])
print("\nlatency_ms → latency_millis kept field_id=4: a rename rewrote NO data.
 ↪")
print("Old rows read back with tier=NULL - no backfill, no migration job.")
```

```
Field IDs before: [(1, 'event_id'), (2, 'ts'), (3, 'model'), (4, 'latency_ms'),
(5, 'cost_usd')]
Field IDs after : [(1, 'event_id'), (2, 'ts'), (3, 'model'), (4,
'latency_millis'), (5, 'cost_usd'), (6, 'tier')]

latency_ms → latency_millis kept field_id=4: a rename rewrote NO data.
Old rows read back with tier=NULL - no backfill, no migration job.
```

[10]:
```
after = tbl.scan().to_arrow()
print(f"Rows still readable: {after.num_rows:,}")
print(f"Columns now: {after.column_names}")
print(f"tier nulls: {after.column('tier').null_count:,} (all pre-existing␣
 ↪rows)")
```

```
Rows still readable: 5,000
Columns now: ['event_id', 'ts', 'model', 'latency_millis', 'cost_usd', 'tier']
tier nulls: 5,000 (all pre-existing rows)
```

1.7 7. Time travel by snapshot

Same idea as Delta's `versionAsOf`, different spelling. Every snapshot is addressable forever until you expire it (that's NB6).

[11]:
```
snap_ids = [s.snapshot_id for s in tbl.snapshots()]
first, last = snap_ids[0], snap_ids[-1]

print(f"snapshot[0]  {first}: {tbl.scan(snapshot_id=first).to_arrow().num_rows:
 ↪,} rows")
print(f"snapshot[-1] {last}: {tbl.scan(snapshot_id=last).to_arrow().num_rows:,}␣
 ↪rows")
print(f"\nTotal snapshots retained: {len(snap_ids)}")

hist = tbl.inspect.history().to_pylist()
print("\nHistory (last 3):")
for h in hist[-3:]:
    print(f"  {h['made_current_at']}  snapshot={h['snapshot_id']}")
```

```
snapshot[0]  7549956316472718628: 500 rows
snapshot[-1] 4623945721596284071: 5,000 rows

Total snapshots retained: 10

History (last 3):
  2026-08-18 16:25:19.514000  snapshot=5507773708506482282
  2026-08-18 16:25:19.558000  snapshot=7480364963616303855
  2026-08-18 16:25:19.611000  snapshot=4623945721596284071
```

1.8 8. Partition evolution — the thing Hive genuinely cannot do

Traffic grew; daily partitions are now too coarse. In Hive this is a rewrite-the-whole-table migration. In Iceberg you change the spec and **old data stays where it is** — each data file remembers which spec wrote it.

[12]:
```
from pyiceberg.transforms import IdentityTransform  # noqa: E402

with tbl.update_spec() as spec:
    spec.add_field("model", IdentityTransform(), "model_id")
tbl = cat.load_table(f"{ns}.llm_events")

tbl.append(day_batch(11).append_column("tier", pa.array(["gold"] *␣
 ↪ROWS_PER_DAY))
           .rename_columns(["event_id", "ts", "model", "latency_millis",␣
 ↪"cost_usd", "tier"]))
tbl = cat.load_table(f"{ns}.llm_events")

specs_in_use = set(tbl.inspect.files().column("spec_id").to_pylist())
```

```
print(f"Partition specs in use across data files: {sorted(specs_in_use)}")
print(f"Total rows readable across BOTH specs: {tbl.scan().to_arrow().num_rows:
  ↪,}")
print("\nTwo layouts, one table, zero rewrites. This is the feature.")
```

```
Partition specs in use across data files: [1, 2]
Total rows readable across BOTH specs: 5,500

Two layouts, one table, zero rewrites. This is the feature.
```

1.9 NB5 pass criteria

Check	Target
Hidden-partition pruning ratio	5× (printed in §4)
Snapshots retained	10
Schema evolved, field IDs stable	`latency_millis` keeps `field_id=4`
Partition evolution	2 distinct `spec_id`s, table still fully readable

[13]:
```
checks = {
    "pruning ratio  5x":        PRUNE_RATIO >= 5,
    "  10 snapshots":           len(tbl.snapshots()) >= 10,
    "field_id stable on rename": [f.field_id for f in tbl.schema().fields if f.
  ↪name == "latency_millis"] == [4],
    "  2 partition specs":      len(specs_in_use) >= 2,
    "all rows readable":        tbl.scan().to_arrow().num_rows == (N_DAYS + 1)␣
  ↪* ROWS_PER_DAY,
}
for k, v in checks.items():
    print(f"  [{'PASS' if v else 'FAIL'}] {k}")
assert all(checks.values()), "NB5 incomplete - see FAIL rows above"
print("\nNB5 complete.")
```

```
  [PASS] pruning ratio  5x
  [PASS]   10 snapshots
  [PASS] field_id stable on rename
  [PASS]   2 partition specs
  [PASS] all rows readable

NB5 complete.
```